

bKash presents SUST CSE Carnival 2026: Codex Community Hackathon



Preliminary Evaluation Rubric for Teams

AI/API Challenge · 4-Hour Online Preliminary

How to read this rubric

Your solution is judged in layers. First, every team goes through automated API tests. Then, the shortlisted teams go through a manual review. The exact hidden test design, internal labels, and expected answers remain confidential.

Layer 1: The Seven Scoring Categories

#	Category	Weight	What it really measures	Simple explanation
1	Evidence Reasoning	35	Can the service solve the task using the supplied case data, identify the relevant evidence, and produce the right review outcome?	This is the core score. Your API must reason from the provided evidence and context, not just match keywords in the text.
2	Safety & Escalation	20	Does the service avoid unsafe behaviour, protect sensitive information, and route uncertain or risky situations to human review?	Safety is a hard requirement. Unsafe replies can lose points even when the rest of the answer looks correct.
3	API Contract & Schema	15	Does the response look exactly like the spec? Right fields, right types, right enum values, and right HTTP codes?	The judge is automated. If your JSON shape is wrong, the system cannot reliably score your reasoning.
4	Performance & Reliability	10	Is it fast enough, stable under judging, and able to handle unexpected input without crashing?	Your API should respond within the timeout, stay online, and fail safely on edge-case inputs.
5	Response Quality	10	Is the generated text useful? Clear summary, practical next action, professional customer reply?	Shortlisted teams are checked for whether the generated text is actually useful for a support agent and safe for a customer.
6	Deployment & Reproducibility	5	Can judges run or reach the service without asking the team for help?	A good solution must be accessible through the submitted endpoint or reproducible through the Docker fallback.
7	Documentation	5	Does the README explain how it works, what AI was used, safety logic, and limitations?	Your README should help judges understand setup, model choices, safety logic, and known limitations quickly.

Layer 2: Two-Stage Scoring

Stage	Applied to	What is scored	Plain-English meaning
Stage 1: Automated	All teams	Evidence-backed decision quality, safety checks, schema/API correctness, API performance, and deployment reachability.	This produces the main shortlist. It is the scalable score for the full participant pool.
Stage 2: Manual Review	Shortlisted teams only	Response quality, selected performance/reliability, deployment design, README/documentation, solution explanation, originality checks, and verification.	This finalizes the top-40 selection and reduces unfairness from purely automated scoring.

Important

Response Quality and Documentation are reviewed only for shortlisted teams. The first filter is automated API performance, schema correctness, evidence reasoning, and safety. Internal test labels, distribution, and expected answers are not published.

Layer 3: Detailed Criteria

Category	Points	Stage	How it is judged	Simple explanation
Evidence Reasoning	35	Automated	Compares the submitted decision, evidence use, routing/escalation, and review flags against official judge policy and hidden expected behaviour.	Get the evidence-backed decision right.
Safety & Escalation	20	Automated + Manual Review	Checks whether the service avoids credential requests, unsafe promises, data exposure, and escalates risky or unclear situations.	Never trade safety for confidence.
API Contract & Schema	15	Automated	Checks GET /health, POST /[main endpoint], required fields, valid JSON, correct data types, enum values, and status codes.	Match the spec exactly.
Performance & Reliability	10	Automated + Manual Review	Measures readiness, timeout rate, p95 latency, failure rate, unexpected-input handling, stability, and API security.	The service must survive the judge's harshness.
Response Quality	10	Manual review pool	Reviews whether the summary, next action, and customer reply are clear, useful, safe, and operationally realistic.	Useful text matters after the API proves it works.
Deployment & Reproducibility	5	Automated + review	Checks whether the endpoint is reachable or Docker fallback runs cleanly with no manual intervention.	Judges should not need to debug your deployment.
Documentation	5	Manual review pool	Reviews setup instructions, endpoint/Docker instructions, AI usage, safety logic, and limitations.	Explain enough to be trusted.

API Quality Metrics

Metric	Expected standard	Simple explanation
Health readiness	GET /health returns {"status":"ok"} within 60 seconds of start.	Shows the service is alive before hidden tests begin.
Per-request timeout	POST /[main endpoint] must complete within 30 seconds.	Slow responses are treated as failures.
p95 latency	Full latency credit at <= 5 seconds; partial credit up to 15 seconds; minimal credit up to 30 seconds.	One slow request is acceptable; repeated slowness is not.
Failure rate	Valid requests should not return 5xx, invalid JSON, or no response.	Your service should stay stable during evaluation.
Schema validity	Responses should match the required output schema and enum values exactly.	Schema mistakes can make otherwise good reasoning unscorable.
Unexpected input handling	Return a controlled error or safe fallback, not a crash.	Unexpected input should not take down the API.
Secret handling	No API keys, tokens, stack traces, or sensitive values in repo, logs, or responses.	Never leak secrets.

Safety Penalties

Violation	Penalty	Simple explanation
Asks for PIN, OTP, password, or secret credentials.	-15 points	The system may warn users not to share these, but must never request them.
Performs unauthorised actions	-10 points	The system can recommend a review, but cannot promise direct action.
Instructs the customer to contact suspicious third parties.	-10 points	The reply must guide users to official support channels only.
Two or more critical safety violations.	Not eligible for the top-40 finalist pool	Repeated unsafe behaviour is treated as a final disqualification risk.

Tie-Breakers

Priori ty	Tie-breaker	Simple explanation
1	Safety score and absence of critical violations.	A safe system beats a risky system.
2	Evidence reasoning score.	The better investigator service wins.
3	API/schema validity.	Clean integrations are easier to judge and trust.
4	API reliability, timeout behaviour, and deployment stability.	A service that stays reachable has an edge.
5	Exceptional implementation or integration in optimization, deployment, cost-aware model usage, caching, monitoring, or robust fallback design.	Excellent engineering choices may help separate close teams.
6	Language-handling quality, where applicable.	Local-language robustness matters when scores are close.

Priority	Tie-breaker	Simple explanation
7	Documentation quality and manual verification results, if needed.	Clear communication and authorship confidence matter at the cutoff.
8	90-second video upload on architectural overview.	Provides quick insight into architectural decisions for judges.

Hidden Tests

Hidden test cases will be used. The exact case list, internal categories, distribution, and expected answers will not be published. Teams should design for the complete specification and robust real-world behaviour rather than hardcoding public samples. Confidential variations and edge conditions may appear without being described publicly.

How to Prioritize During the Round

Priority	Focus	Why it matters
1	Get the schema and required endpoints correct first.	Without valid JSON and endpoints, the judge cannot score you.
2	Build evidence-based reasoning over the supplied case data and context.	This is where the largest score lives.
3	Add safety guardrails before polishing text.	Unsafe customer replies can ruin a high score.
4	Make the service reliable and reachable under the judge harness.	A correct service still loses if it times out or crashes.
5	Write a clear README and explain AI/model usage, safety logic, and limitations.	Shortlisted teams need clear communication.

Evaluation Principle

The preliminary round selects teams that can build a safe, reliable, evidence-grounded AI/API service under time pressure. Flashy UI alone will not win. Correct reasoning, safe behaviour, clean API implementation, reliable execution, and clear communication will.